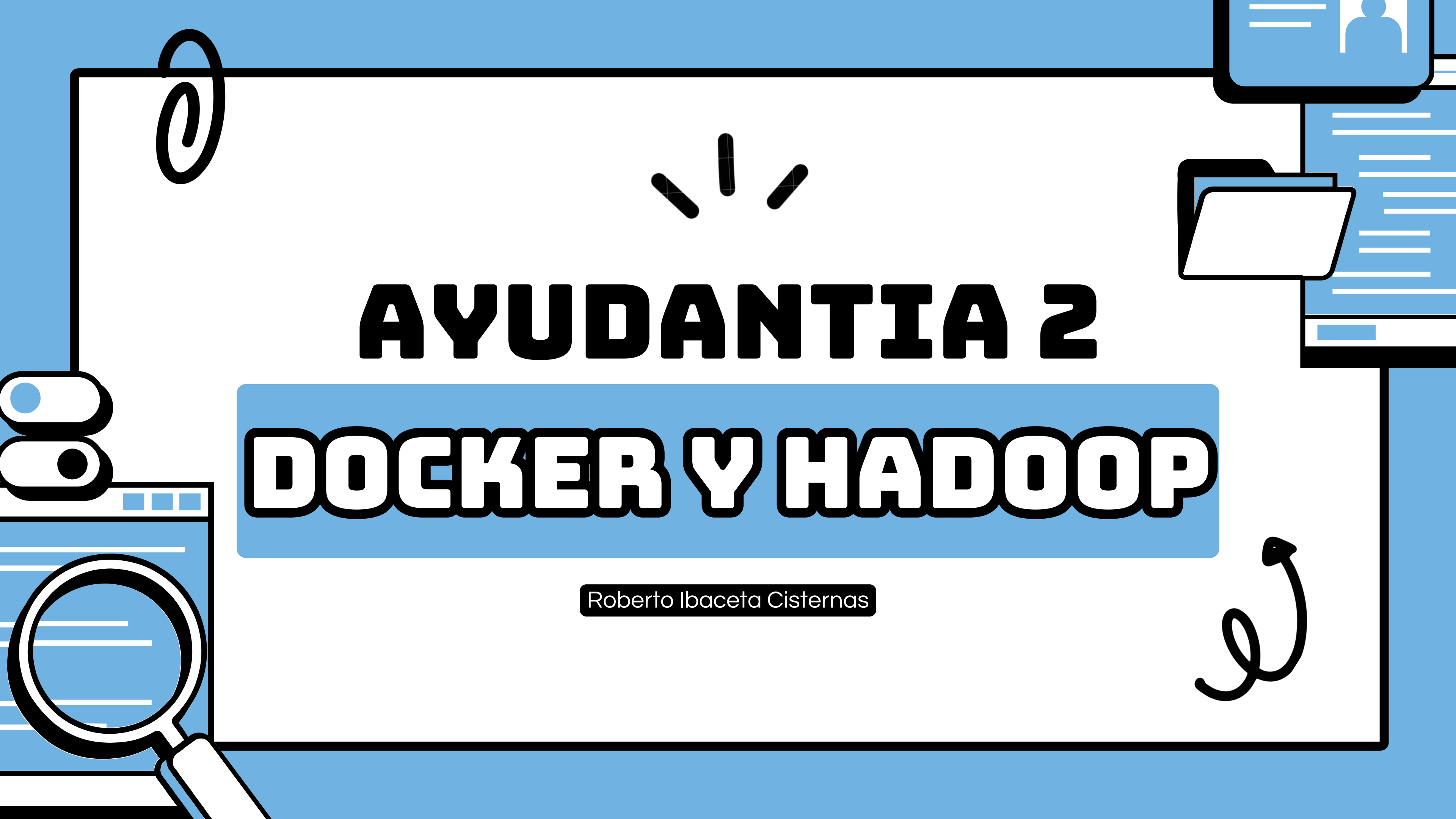


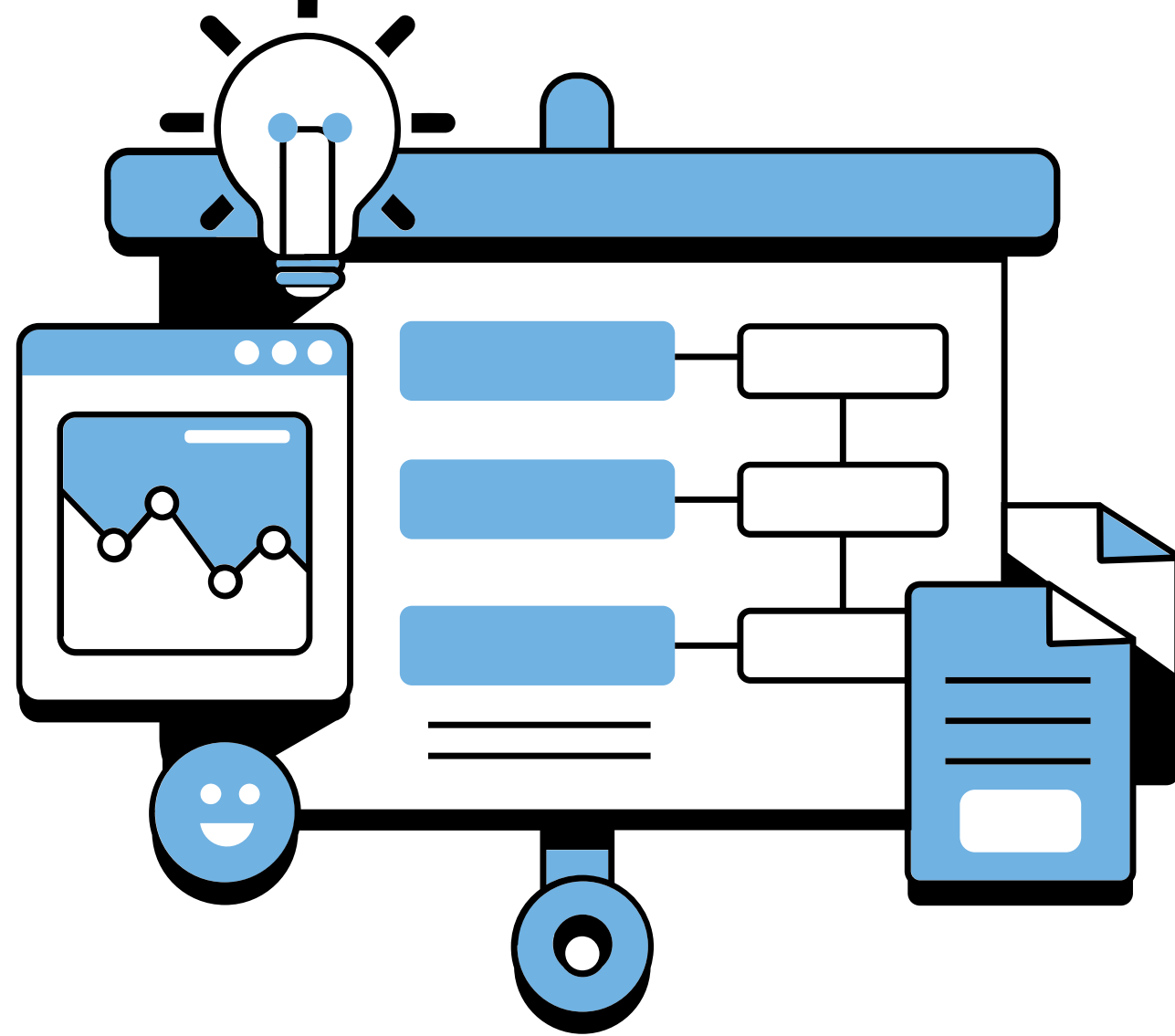


AYUDANTIA 2

DOCKER Y HADOOP

Roberto Ibaceta Cisternas

QUE VEREMOS

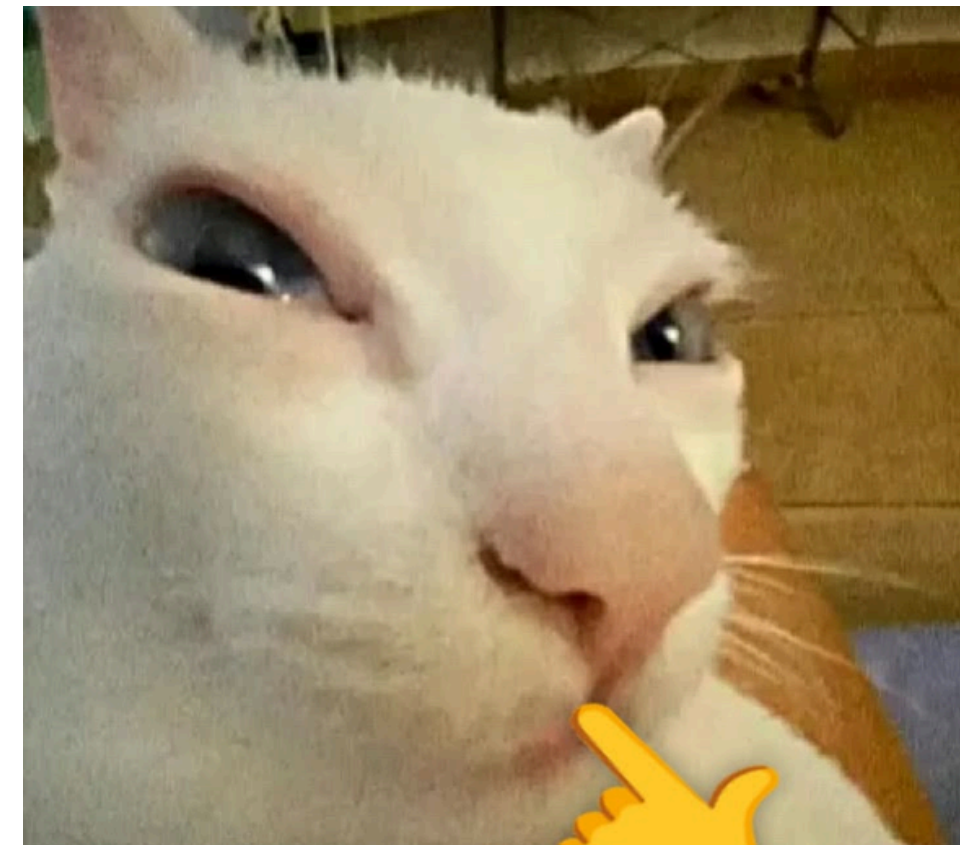


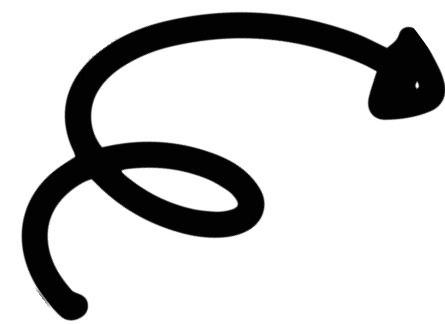
01. Que es Docker?

02. ¿Por qué Hadoop/HDFS?

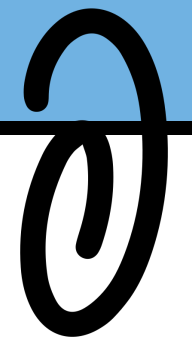
03. Verificación de entorno

04. Levantando Hadoop y operando HDFS





EL LÍMITE DEL SERVIDOR ÚNICO



Escalamiento

Vertical (Scale-Up):

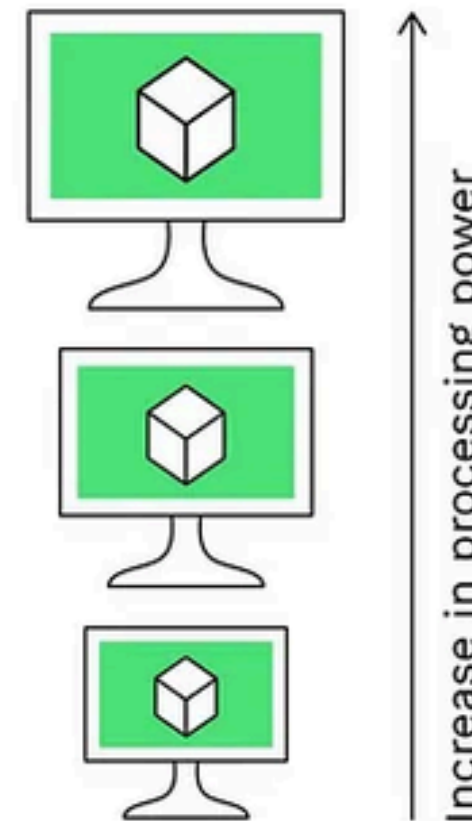
- Un servidor más grande :
(costo exponencial, límite físico de disco/RAM).

Horizontal (Scale-Out):

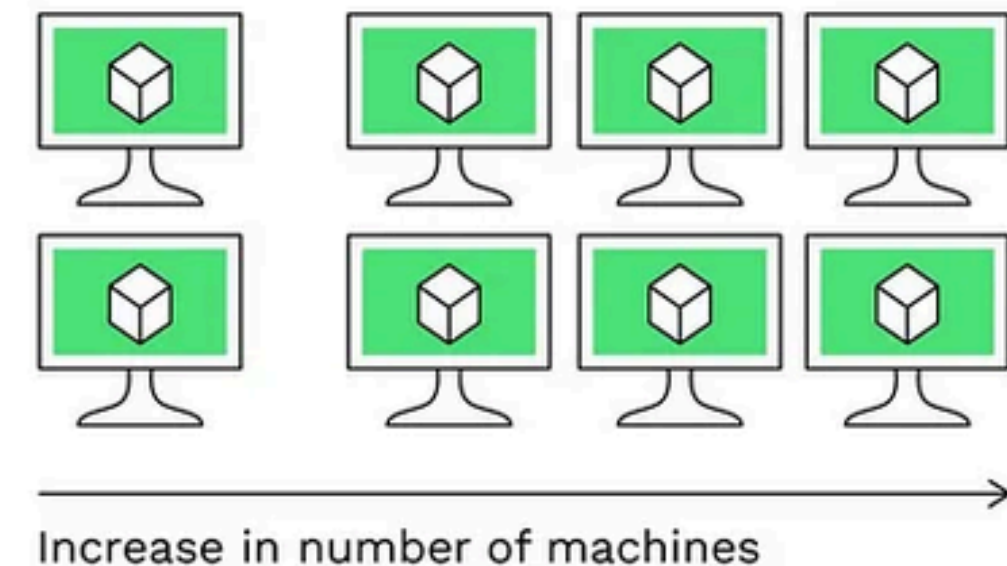
- Muchas máquinas estándar trabajando en equipo
(filosofía Hadoop / Cluster).

Scalability

Vertical scaling



Horizontal scaling



HDFS: BLOQUES Y REPLICACIÓN

Hadoop Distributed File System

Bloques:

- *Los archivos no se guardan enteros; se dividen en bloques de (128 MB vs los 4kb tradicional)*

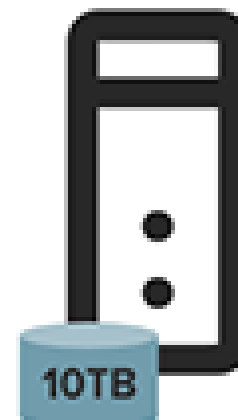
Replicación:

- *Cada bloque se copia en 3 nodos diferentes (Default =3)*

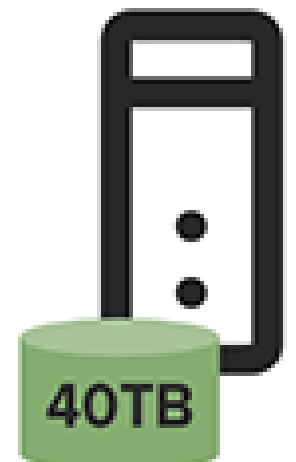
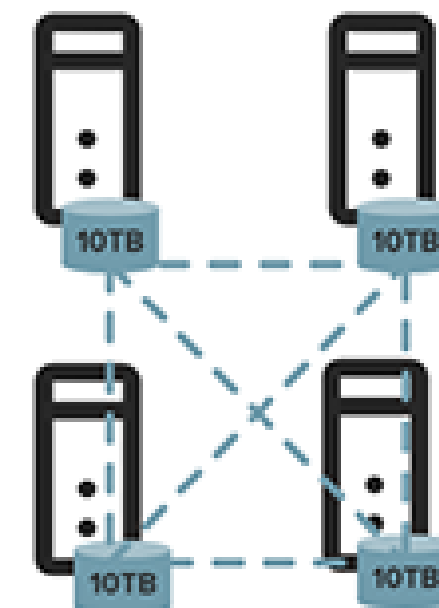
Tolerancia:

- Si un servidor muere, la información no se pierde.

Local File System



DFS (Distributed File System)



ARQUITECTURA HDFS + MAPREDUCE

NameNode:

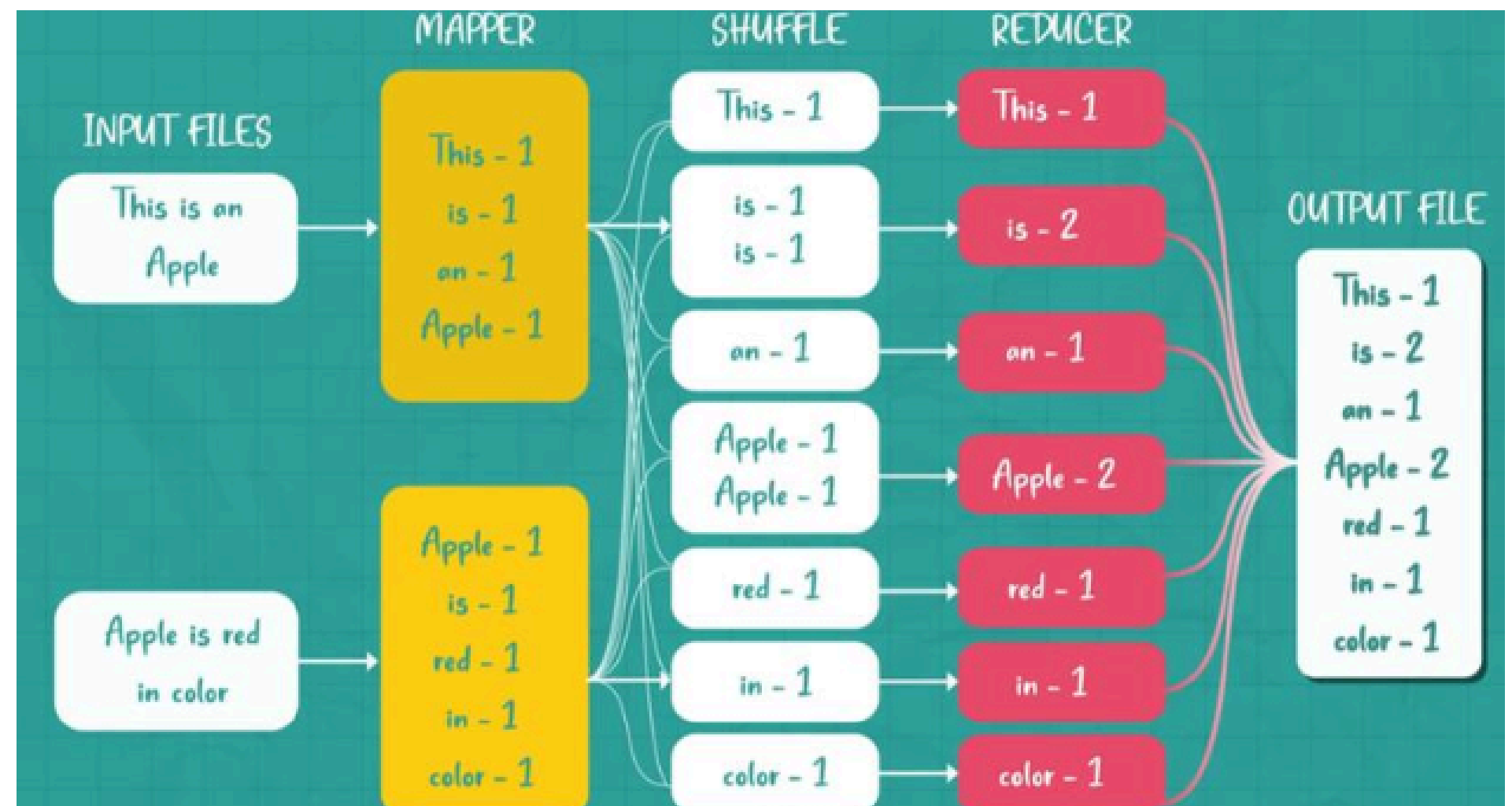
- Guarda el mapa de metadatos: *sabe exactamente en qué nodo está cada bloque.*
- No almacena datos de usuario, solo la estructura.

Data Nodes:

- Almacenan y leen los bloques físicos en disco, **(128 MB)**.
- Reportan constantemente su estado de salud al NameNode.

MapReduce:

- Mover el Código, NO los Datos:
 - Llevar el código a los datos (**no mover terabytes por la red**).
 - • Map: procesa en paralelo en cada nodo.
 - • Reduce: consolida resultados finales



EJEMPLO

Biblioteca:

- Imaginen una Biblioteca enorme (**nuestros datos**), En vez de guardar cada libro completo en un estante (**un Disco**). **HDFS** arranca cada libro en capítulos (**bloques de 128 MB**), y los reparte en distintas salas (**servidores**), y ademas saca 2 fotocopias de cada capitulo, por si una sala se incendia (**Se cae**). El **NameNode** es el bibliotecario, que sabe donde esta cada capitulo (**El Gato**).

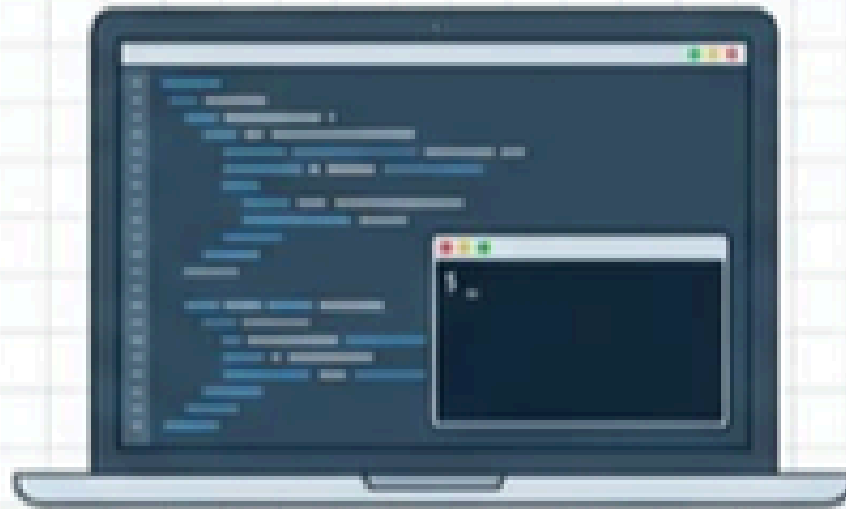


POR QUÉ DOCKER?

El Problema Universal: "Funciona en mi máquina"

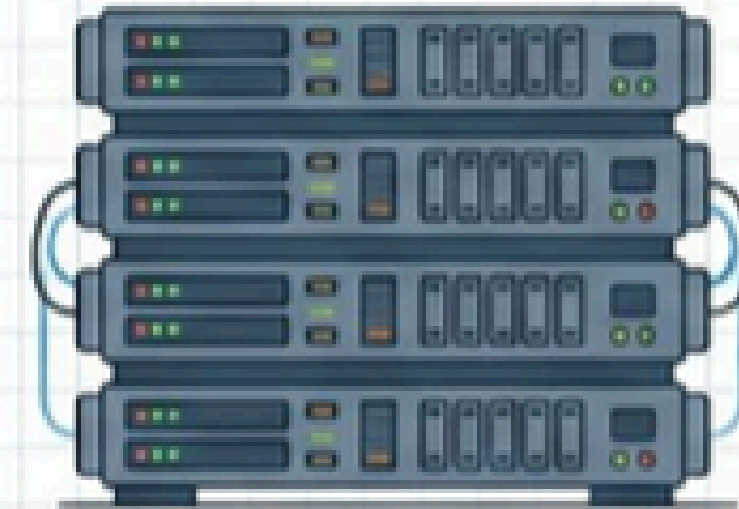
Tu aplicación corre perfectamente en tu portátil, pero falla estrepitosamente en el servidor de producción. ¿Por qué?

Entorno Local

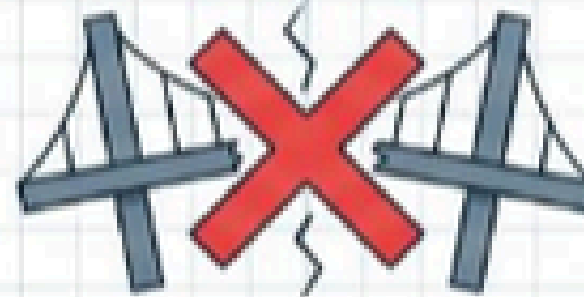


- ❑ - OS: Ubuntu 22.04
- ❑ - Librerías: v1.2 (Instaladas a mano)
- ❑ - Configuración: Variables de entorno locales

Producción



- ❑ - OS: CentOS 7
- ❑ - Librerías: v1.0 (Versión antigua)
- ❑ - Configuración: Faltan dependencias clave



El código no es el problema; el problema es la inconsistencia del entorno.

La Solución: Contenedores

Un contenedor empaqueta una aplicación y todas sus dependencias en una unidad estandarizada.



Ligero

Comparten el kernel del sistema anfitrión (host). No requieren un sistema operativo completo para cada aplicación.



Portable

Se ejecutan de manera idéntica sin importar dónde se desplieguen (tu portátil, un servidor físico o la nube).

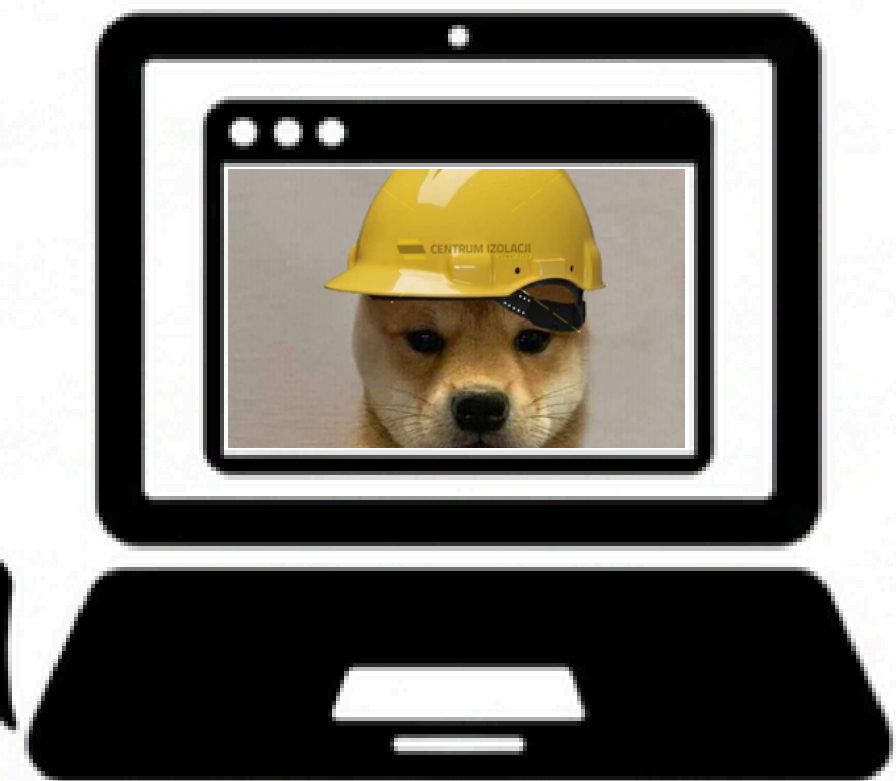


Aislado

Se ejecutan en procesos independientes. No interfieren con otras aplicaciones en el mismo sistema.

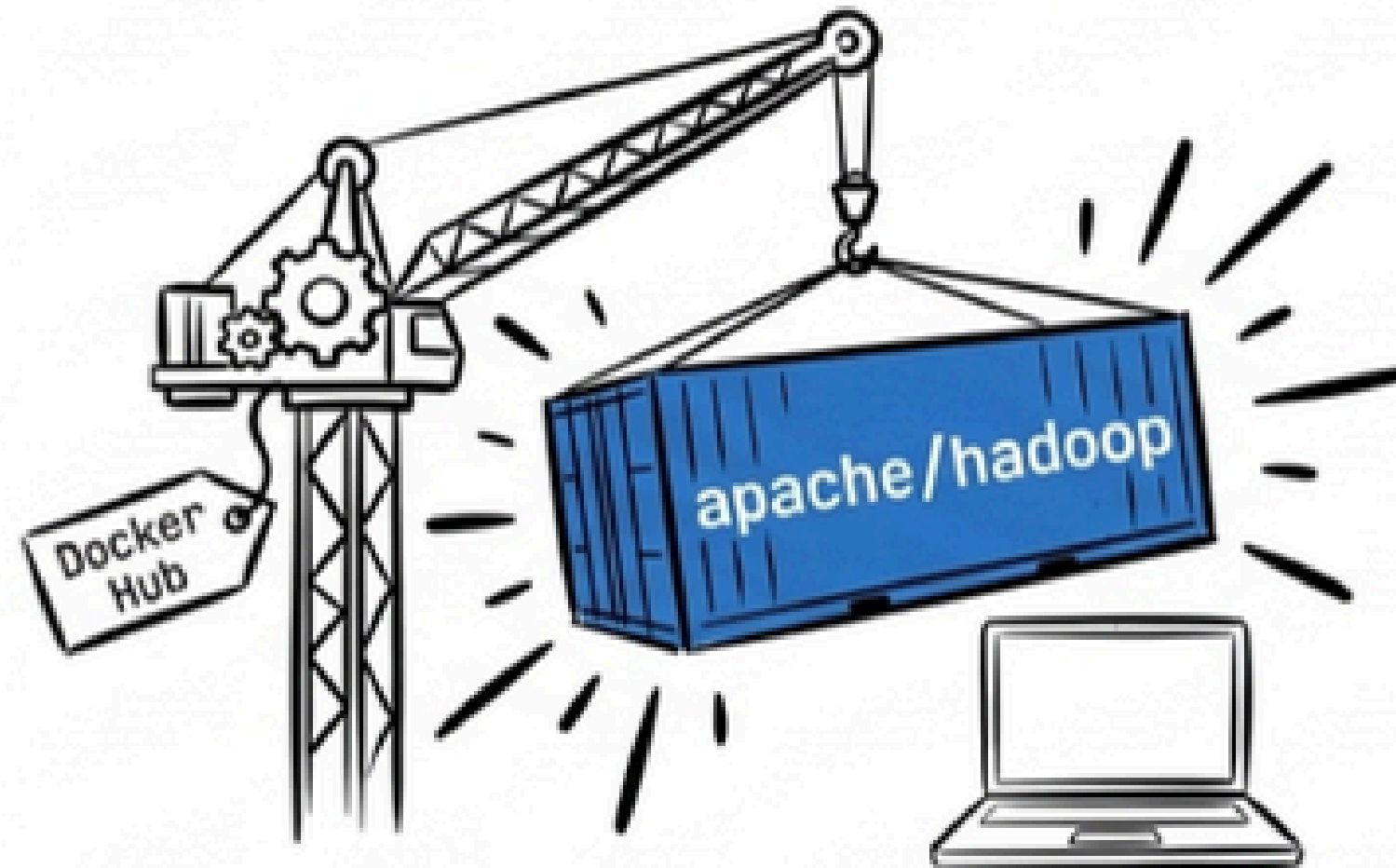
MANOS A LA OBRA

Levantando Hadoop
en tu propia máquina.



Paso 1: Descargar la Imagen

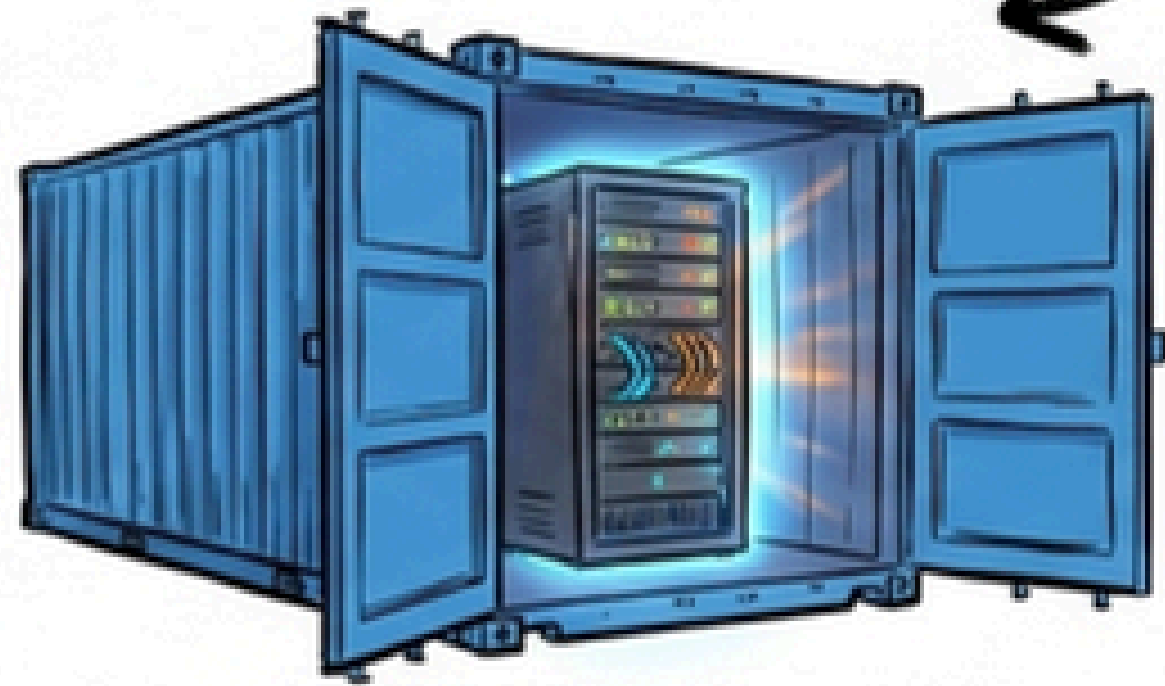
```
Terminal Window  
> docker pull apache/hadoop
```



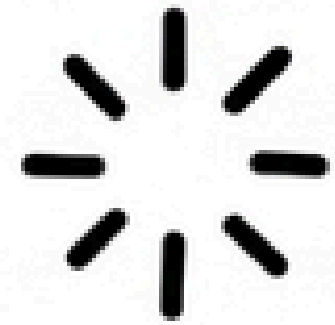
Descargamos la plantilla oficial y empaquetada.
Cero configuraciones manuales de Java o variables de entorno.

Paso 2: Iniciar Contenedor Interactivo

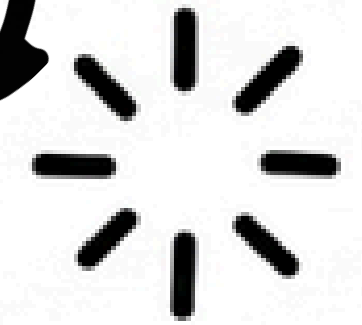
```
> docker run -it apache/hadoop
```

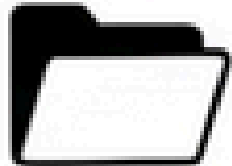
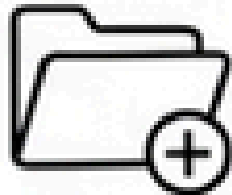
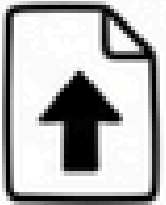
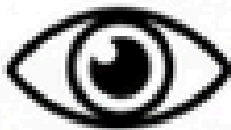


Levanta el contenedor basado
Terminal interactiva. Te
mete "dentro" del
sistema Linux virtualizado.



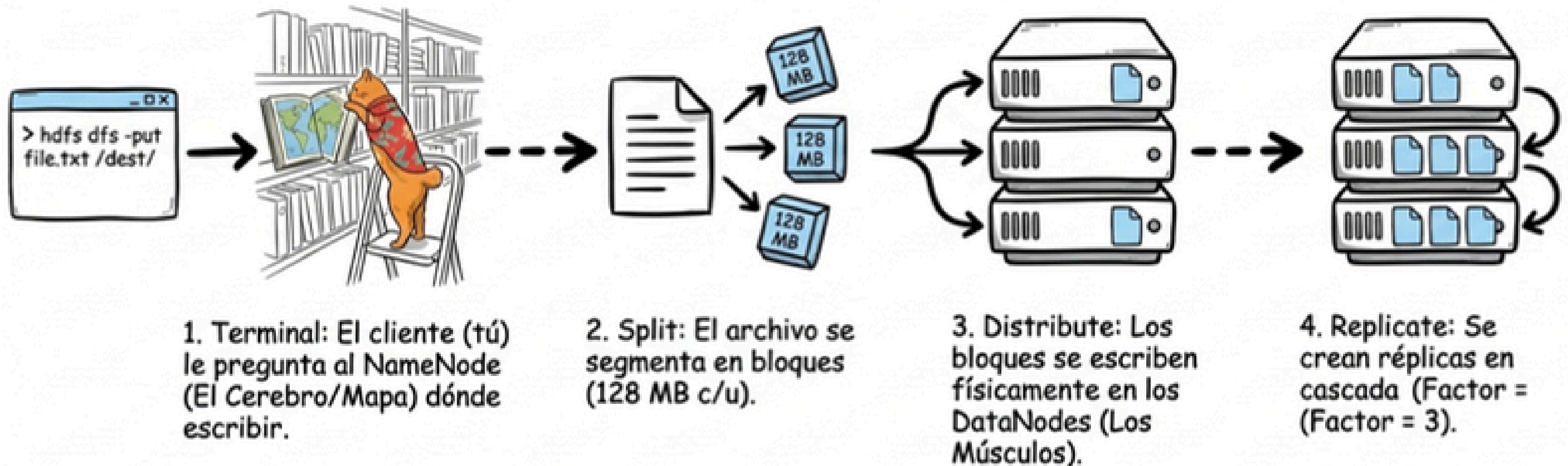
Comandos Esenciales de HDFS (Cheat Sheet)



Acción	Linux Tradicional	Mundo HDFS
	ls	<code>hdfs dfs -ls /</code>
	mkdir	<code>hdfs dfs -mkdir -p /user/datos</code>
	cp/mv	<code>hdfs dfs -put archivo.txt /user/datos/</code>
	cat	<code>hdfs dfs -cat /user/datos/archivo.txt</code>

QUE PASO REALMENTE?

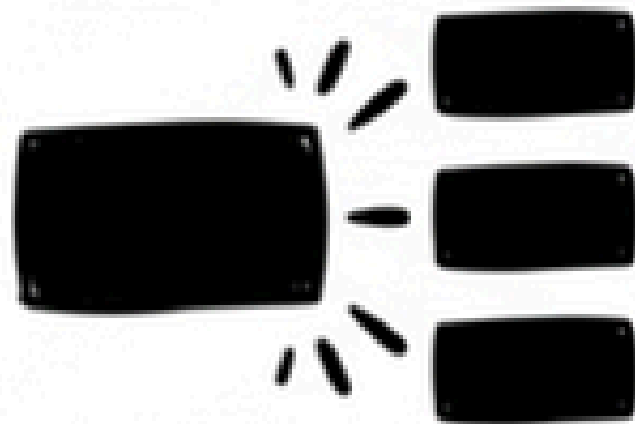
La magia detrás de un simple comando put.



3 Ideas Clave para Llevarse a Casa



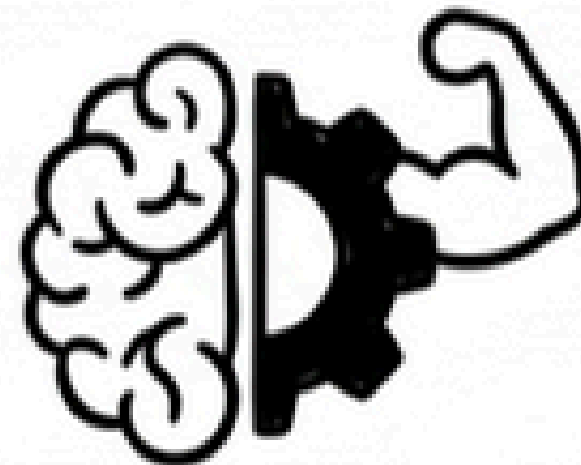
01



HDFS divide y replica

Bloques grandes (128 MB) y 3 copias por defecto para tolerancia total a fallos.

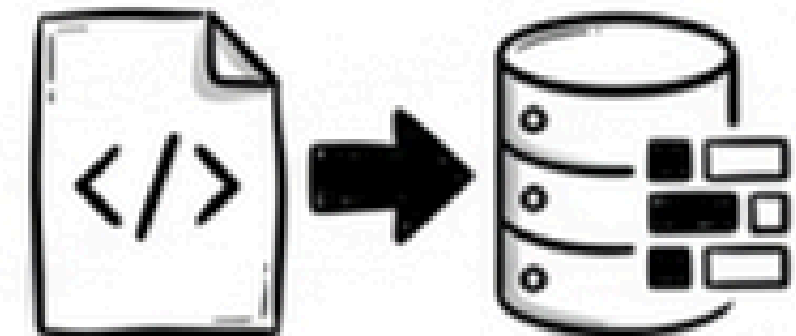
02



Arquitectura Separada

NameNode guarda el mapa de metadatos; los DataNodes almacenan los bytes reales.

03



MapReduce

Cómputo donde están los datos. Es más rápido mover el código (KB) que mover los datos (TB) por la red.

BONUS

Pregunta 1 (1 punto)

Si una base de datos necesita leer 1,000 registros dispersos aleatoriamente en un disco duro mecánico (HDD), el rendimiento se desploma drásticamente comparado con leer un archivo de forma secuencial. Según lo visto en clases, ¿cuál es la razón física exacta de este fenómeno?

- a) El disco duro lee los datos byte por byte en lugar de usar bloques paginados, sobrecargando el bus de datos.
- b) El sistema operativo debe ordenar los registros en la memoria RAM con una complejidad $O(N \log N)$ antes de entregarlos.
- c) El brazo mecánico del disco debe realizar un movimiento físico (Seek Time) para encontrar la pista de *cada* bloque individual, lo cual destruye el rendimiento mecánico.
- d) Las garantías ACID obligan a que el disco duro detenga su rotación temporalmente para asegurar la consistencia.

Si una base de datos necesita leer 1,000 registros dispersos aleatoriamente en un disco duro mecánico (HDD), el rendimiento se desploma drásticamente comparado con leer un archivo de forma secuencial. Según lo visto en clases,

¿cuál es la razón física exacta de este fenómeno?

a) El disco duro lee los datos byte por byte en lugar de usar bloques paginados, sobrecargando el bus de datos.

- Es falsa, ya que los Discos Nunca leen bytes, siempre leen bloques

b) El sistema operativo debe ordenar los registros en la memoria RAM con una complejidad $O(N \log N)$ antes de entregarlos.

- La RAM no debiera por que estar involucrada en esta lectura, ademas si lo estuviera, lo haria muchisimo mas rapido sin el desplome

c) ***El brazo mecánico del disco debe realizar un movimiento físico (Seek Time) para encontrar la pista de cada bloque individual, lo cual destruye el rendimiento mecánico. entregarlos.***

- Esta es la razon fisica real, al ser un disco HDD y tener que leer 1.000 datos desordenados, implica que el brazo tenga que moverse las 1.000 veces a un lugar distinto.

d) Las garantías ACID obligan a que el disco duro detenga su rotación temporalmente para asegurar la consistencia.

- ACID, es una garantia a nivel motor de base de datos, no algo que controle a un disco duro.



Pregunta 3 (4 puntos)

Contexto: Eres el Ingeniero de Datos en una startup. La base de datos relacional de la empresa ha crecido a 50 Terabytes. El equipo de software ha hecho un trabajo perfecto: todas las consultas utilizan índices B-Tree, logrando tiempos de búsqueda logarítmicos muy eficientes.

Sin embargo, el tráfico ha aumentado a 50,000 transacciones simultáneas por segundo. El servidor actual está colapsando. El gerente general te dice: *"¡El software está optimizado! Solo necesitamos comprar un servidor de un millón de dólares con el doble de CPU y la RAM más rápida del mercado"*.

Tu tarea: Como ingeniero, sabes que este "Escalamiento Vertical" eventualmente fallará. Basándote en los conceptos vistos en clase, responde de forma estructurada:

1. ¿Por qué el sistema B-Tree falla a una escala masiva de transacciones simultáneas en relación con las Garantías ACID?
2. ¿Qué límite físico del hardware hace imposible que un solo servidor soporte este volumen de tráfico, sin importar cuánto dinero gasten?
3. Arquitectónicamente, ¿cuál es la única solución técnica viable para superar este límite de una sola máquina?

Contexto: Eres el Ingeniero de Datos en una startup. La base de datos relacional de la empresa ha crecido a 50 Terabytes. El equipo de software ha hecho un trabajo perfecto: todas las consultas utilizan índices B-Tree, logrando tiempos de búsqueda logarítmicos muy eficientes. Sin embargo, el tráfico ha aumentado a 50,000 transacciones simultáneas por segundo. El servidor actual está colapsando. El gerente general te dice:

"¡El software está optimizado! Solo necesitamos comprar un servidor de un millón de dólares con el doble de CPU y la RAM más rápida del mercado"

¿Por qué el sistema B-Tree falla a una escala masiva de transacciones simultáneas en relación con las Garantías

- ACID?*
- **Eficiencia vs. Concurrencia:** Un índice B-Tree optimiza la búsqueda $O(\log n)$, pero no resuelve los cuellos de botella por alta concurrencia de escritura.
 - **Garantías ACID (Aislamiento y Consistencia):** Exigen mecanismos como bloqueos (locks a nivel de fila/página) o control por versiones (MVCC).
 - **Contención de Locks:** Cuando miles de transacciones escriben simultáneamente sobre las mismas páginas o nodos del árbol, deben serializarse y esperar su turno.
 - **Conclusión:** A gran escala, la degradación no se debe a la complejidad algorítmica, sino al costo de sincronización y coordinación transaccional.

¿Qué límite físico del hardware hace imposible que un solo servidor soporte este volumen de tráfico, sin importar cuánto dinero gasten?

- **Cuello de Botella en E/S (I/O de Disco):** El límite crítico son los IOPS y el ancho de banda del subsistema de almacenamiento, no la potencia de cálculo (CPU).
- **Garantía de Durabilidad (ACID):** Cada transacción confirmada obliga a persistir datos físicamente en disco, impidiendo resolver el problema solo con RAM rápida.
- **Límites de Red y Conexiones:** Existe un tope físico de conexiones simultáneas y ancho de banda de red que una sola placa/interfaz puede procesar por segundo.
- **Rendimientos Decrecientes (Ley de Amdahl):** Agregar más hardware a un único nodo pierde efectividad, ya que tareas críticas (como los commits a disco y la sincronización de locks) son inherentemente secuenciales y no se pueden paralelizar al infinito.

Arquitectónicamente, ¿cuál es la única solución técnica viable para superar este límite de una sola máquina?

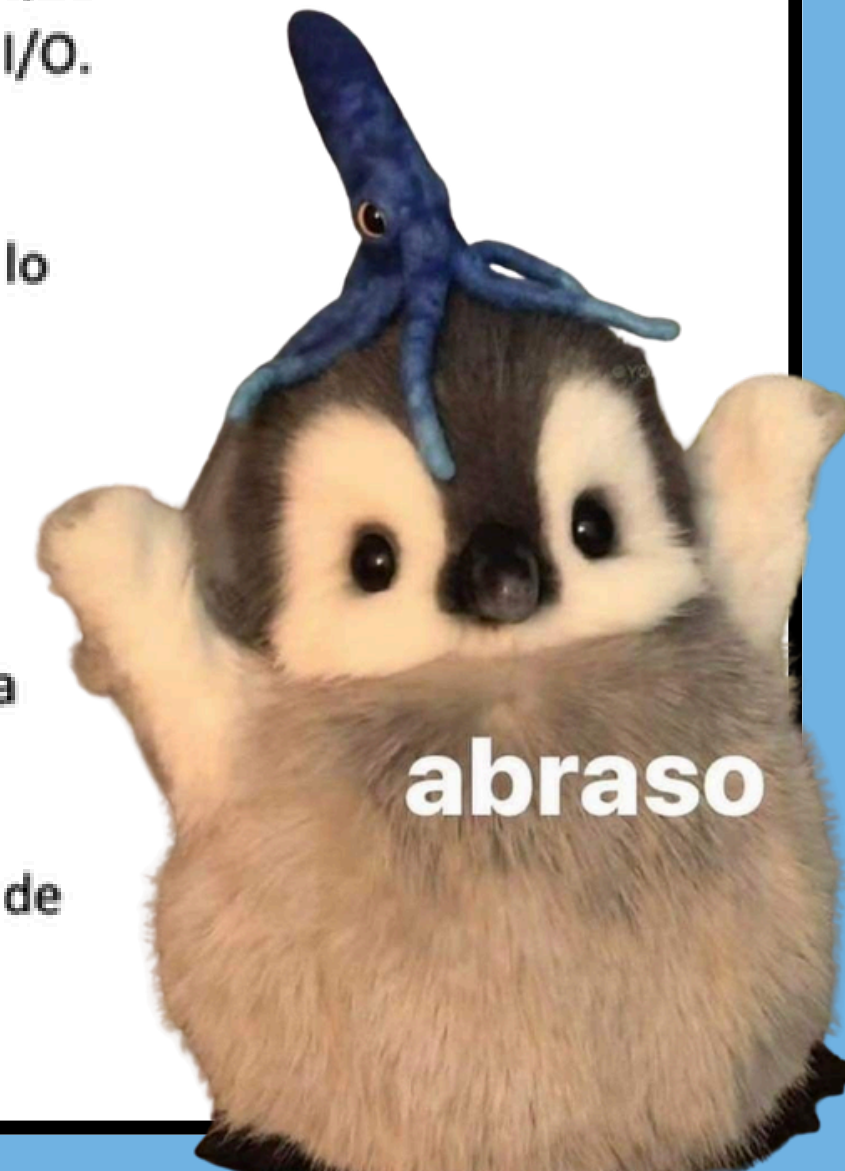
Idea clave para retener

El punto central de esta pregunta es distinguir entre optimización algorítmica (el B-Tree, que ya está bien hecho) y límites físicos de una sola máquina (I/O, concurrencia, IOPS). Ningún algoritmo eficiente por sí solo resuelve un problema de escala de infraestructura: en algún punto, la única solución es distribuir la carga entre múltiples máquinas.

Pregunta 2 (1 punto)

Acabas de realizar una carga masiva de datos (Bulk Insert) en una tabla que tiene un índice B-Tree configurado. Notas que a medida que insertas más datos, la operación se vuelve más lenta y el disco reporta una alta latencia de operaciones I/O. ¿Qué está ocurriendo físicamente en la estructura del índice?

- a) El B-Tree está realizando operaciones de "Node Splitting" (división de nodos) porque los bloques se llenan, lo que requiere asignar nuevos bloques y mover datos.
- b) El B-Tree se está transformando en un árbol binario estándar debido a la cantidad de datos, aumentando drásticamente la altura del árbol.
- c) La CPU está esperando a que la memoria caché L1 se sincronice con el disco mecánico antes de confirmar la transacción.
- d) El motor de base de datos está haciendo un Full Scan para verificar que no haya registros duplicados antes de cada inserción.



CONTACTO

Gmail

- roberto.ibaceta@mail.udp.cl

Wsp

- [+569 42663436](https://wa.me/56942663436)

Ds

- [roberibaz](#)

